EXPENSE TRACKER APPLICATION

Using CRUD Operations, Categories and State Management

Name : Mistry Anshu — Enrollment No.: 2305101270447

Parmar Bhavik — Enrollment No.: 2305101270205

1. Introduction

- The project is a simple Expense Tracker application designed to record and manage daily expenses.
- Users can add new expenses with a description, amount and category.
- Existing expenses can be deleted when they are no longer needed.
- Expenses can be organized by categories such as Food, Travel, Shopping and Other.
- A running total is displayed so users can quickly see their current spending.

2. Objectives

- Add and store expense records with basic details.
- Delete an expense from the current list.
- Categorize expenses for easier organization and viewing.
- Calculate and display a running total of all expenses.
- Understand basic CRUD operations and state management through a practical application.

3. Components / Features

Application Features

- Expense input form
- Expense list / history
- Category selection
- Delete expense action
- Running total display

Core Concepts

- Create expense records
- Read and display records
- Update application state
- Delete selected records
- Calculate derived total

4. Working Principle

- 1. The user enters an expense description, amount and category.
- 2. The application creates a new expense record and adds it to the expense list.
- 3. The updated list is displayed immediately.
- 4. The running total is recalculated from the current expense records.
- 5. The user can filter or review expenses by category.
- 6. When an expense is deleted, the list and running total update again.

5. Expense Data Structure

- Each expense can be represented as a simple record/object.
- id → unique identifier for the expense.
- description → short name or note for the expense.
- amount → numeric value of the expense.
- category → group such as Food, Travel, Shopping or Other.
- date → date on which the expense was recorded.

6. Algorithm / Logic

- Start the application.
- Initialize the expenses list and total.
- Accept description, amount and category from the user.
- Create a new expense object and add it to the list.
- Recalculate the running total from all current expenses.
- Display the expense list and total.
- If the user deletes an expense, remove it and recalculate the total.
- Repeat whenever the user adds, views or deletes an expense.

8. Important Code Logic

```
addExpense(expense):
    expenses = [...expenses, expense]
    total = expenses.reduce((sum, e) => sum + e.amount, 0)

deleteExpense(id):
    expenses = expenses.filter(e => e.id !== id)

filterByCategory(category):
    return expenses.filter(e => e.category === category)
```

9. Expected Output

Total Expenses	Food	Travel	Other
■ 2,450	■ 1,200	■ 800	■ 450

Description	Category	Amount
Lunch	Food	■ 300
Bus Pass	Travel	■ 500
Groceries	Food	■ 900
Stationery	Other	■ 450

The running total updates whenever an expense is added or deleted.

10. Applications

- Personal daily expense tracking
- Student pocket-money management
- Household spending records
- Small project or event budget tracking
- Learning CRUD operations, lists and state management
- Prototype for larger finance-management applications

11. Advantages & Limitations

Advantages

- Simple and easy to use
- Quick expense entry
- Automatic running total
- Categories improve organization
- Good demonstration of CRUD + state management

Limitations

- Basic tracker without advanced budgeting
- No automatic bank transaction import
- Data persistence depends on the implementation
- No authentication or multi-user support
- Reports and analytics are limited

12. Conclusion

- The project demonstrates a practical Expense Tracker using basic CRUD operations.
- Users can add and delete expense records while keeping the list updated.
- Categories make expense records easier to organize and review.
- State management keeps the displayed list and running total synchronized.
- The project provides a simple foundation for extending the application with storage, reports and budgeting features.

13. References

- CRUD (Create, Read, Update, Delete) application concepts
- State management and derived-state calculation concepts
- Basic list filtering and aggregation concepts
- Project source code and user-interface implementation
- Expense Tracker project requirements provided for the CPD mini project